

LIBRARY MANAGEMENT SYSTEM



File Handling Based Project in C++

SUBMITTED BY:

Laiba Wajid: 62 Zaiba khaleel: 82

Esha Younas: 72 Sohail Hamdani: 92

SECTION: BSCS (B)

SEMESTER: 1ST

SESSION: 2025 – 29

SUBMITTED TO:

DR. WAJID AZIZ

1. Introduction

The **Library Management System** is a console-based application developed in **C++** using **file handling concepts**.

This system is designed to manage library records efficiently by allowing the user to **add, view, search, and delete book records**.

The main purpose of this project is to reduce manual work, improve data accuracy, and provide an easy way to store and retrieve book information from a file.

2. Objectives of the Project

The main objectives of this project are:

- To maintain book records permanently using files
 - To add new books to the library system
 - To display all available books
 - To search a book using its unique Book ID
 - To delete a book record when required
 - To understand the practical implementation of **file handling in C++**
-

3. Scope of the Project

This project is suitable for **small-scale libraries, departmental libraries, or student projects**. It provides basic library operations and can be further enhanced by adding features like book issuing dates, return dates, or fine calculation.

4. Tools and Technologies Used

Tool / Technology	Description
Programming Language	C++
IDE / Compiler	Dev C++ / Code::Blocks / Visual Studio

File Handling	Binary File (books.dat)
Operating System	Windows

5. Data Structure Used

- A **structure** named `Book` is used to store book information:
 - **Attributes of Book Structure:**
 - Book ID
 - Book Name
 - Author Name
 - Student Name (if issued)
 - Book Price
 - Number of Pages
 - This structure allows storing complete book information in a single record.
-

6. Functional Description

6.1 Add Book

This function allows the user to add a new book record.
The entered data is stored permanently in a binary file named **books.dat**.

Input:

- Book ID
 - Book Name
 - Author Name
 - Student Name
 - Price
 - Pages
-

6.2 Display All Books

This function reads all records from the file and displays them on the screen in a well-formatted manner.

Purpose:

- View all books available in the library
-

6.3 Search Book by ID

This function searches a specific book using its **Book ID**. If the book exists, its complete details are displayed.

Purpose:

- Fast and accurate searching of books
-

6.4 Delete Book

This function deletes a book record using **Book ID**. The data is copied to a temporary file except the deleted record, and then the original file is updated.

Purpose:

- Remove outdated or incorrect records
-

7. File Handling Technique

- Binary files are used for data storage
- `ofstream` is used to write data
- `ifstream` is used to read data
- `ios::binary` mode ensures efficient storage

The file name used is **books.dat**.

8. Menu-Driven Interface

The program uses a **menu-driven approach**, making it easy for users to interact with the system.

Menu Options:

1. Add Book
 2. View All Books
 3. Search Book by ID
 4. Delete Book
 5. Exit
-

9. Advantages of the System

- Easy to use
 - Permanent data storage
 - Fast searching
 - Reduces paperwork
 - Simple and user-friendly interface
-

10. Limitations of the System

- No graphical user interface
 - No password or security system
 - Limited to basic operations
 - Works on single file only
-

11. Future Enhancements

The following features can be added in the future:

- Book issue and return date
 - Fine calculation system
 - Login system for admin and users
 - Update book record option
 - Graphical User Interface (GUI)
-

12. Conclusion

PROJECT

The **Library Management System** is a simple and efficient C++ project that demonstrates the use of **structures, file handling, and menu-driven programming**.

This project fulfills all basic library management requirements and serves as a strong foundation for advanced systems.

13. References

- C++ Programming Language (Books & Notes)
- Online C++ Documentation
- Lecture Notes Provided by Instructor

14. Source Code

Below is the complete source code of the **Library Management System** developed in C++ using file handling.

This code implements all basic library operations including adding, viewing, searching, and deleting book records.

Library Management System (C++ Source Code)

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <string>
```

```
using namespace std;
```

```
//***** Structure to store book information*****
```

```
struct Book {
```

```
    int bookID;
```

```
    string bookName;
```

```
    string author;
```

```
    string studentName;
```

```
    float price;
```

```
    int pages;
```

PROJECT

```
};
```

```
//***** Function to add a book to the file*****
```

```
void addBook() {  
    Book book;  
    ofstream file("books.dat", ios::app | ios::binary);  
    if (!file) {  
        cout << "Error opening file!" << endl;  
        return;  
    }  
    cout << "Enter Book ID: ";  
    cin >> book.bookID;  
    cin.ignore();  
    cout << "Enter Book Name: ";  
    getline(cin, book.bookName);  
    cout << "Enter Author Name: ";  
    getline(cin, book.author);  
    cout << "Enter Student Name (if issued): ";  
    getline(cin, book.studentName);  
    cout << "Enter Book Price: ";  
    cin >> book.price;  
    cout << "Enter Number of Pages: ";  
    cin >> book.pages;  
    cin.ignore();
```

PROJECT

```
file.write((char*)&book, sizeof(Book));

file.close();

cout << "Book added successfully!" << endl;

}

//***** Function to display all books*****

void displayBooks() {

    Book book;

    ifstream file("books.dat", ios::binary);

    if (!file) {

        cout << "No books available or error opening file!" << endl;

        return;

    }

    cout << "\nList of Books:\n";

    while (file.read((char*)&book, sizeof(Book))) {

        cout << "-----\n";

        cout << "Book ID: " << book.bookID << endl;

        cout << "Book Name: " << book.bookName << endl;

        cout << "Author: " << book.author << endl;

        cout << "Student Name (if issued): " << book.studentName << endl;

        cout << "Price: " << book.price << endl;

        cout << "Pages: " << book.pages << endl;

    }

    cout << "-----\n";

    file.close();

}
```


PROJECT

```
}
```

```
//***** Function to search a book by ID*****
```

```
void searchBookByID() {
```

```
    Book book;
```

```
    int bookID;
```

```
    ifstream file("books.dat", ios::binary);
```

```
    if (!file) {
```

```
        cout << "Error opening file!" << endl;
```

```
        return;
```

```
    }
```

```
    cout << "Enter Book ID to search: ";
```

```
    cin >> bookID;
```

```
    bool found = false;
```

```
    while (file.read((char*)&book, sizeof(Book))) {
```

```
        if (book.bookID == bookID) {
```

```
            cout << "\nBook Found:\n";
```

```
            cout << "Book ID: " << book.bookID << endl;
```

```
            cout << "Book Name: " << book.bookName << endl;
```

```
            cout << "Author: " << book.author << endl;
```

```
            cout << "Student Name (if issued): " << book.studentName << endl;
```

```
            cout << "Price: " << book.price << endl;
```

```
            cout << "Pages: " << book.pages << endl;
```

```
            found = true;
```

PROJECT

```
        break;
    }
}

if (!found) {
    cout << "Book with ID " << bookID << " not found." << endl;
}

file.close(); }

//*****Function to delete a book by ID*****

void deleteBook() {
    Book book;
    int bookID;

    ifstream file("books.dat", ios::binary);
    ofstream temp("temp.dat", ios::binary);
    if (!file || !temp) {
        cout << "Error opening file!" << endl;
        return;
    }

    cout << "Enter Book ID to delete: ";
    cin >> bookID;

    bool found = false;
    while (file.read((char*)&book, sizeof(Book))) {
        if (book.bookID != bookID) {
            temp.write((char*)&book, sizeof(Book));
        } else {
```

PROJECT

```
        found = true;
    }
}

file.close();

temp.close();

remove("books.dat");

rename("temp.dat", "books.dat");

if (found) {

    cout << "Book with ID " << bookID << " has been deleted successfully." << endl;

} else {

    cout << "Book with ID " << bookID << " not found." << endl;

}

}

//***** Main function*****

int main() {

    int option;

    do {

        cout << "\nLibrary Management System\n";

        cout << "1. Add Book\n";

        cout << "2. View All Books\n";

        cout << "3. Search Book by ID\n";

        cout << "4. Delete Book by ID\n";

        cout << "5. Exit\n";

        cout << "Enter your choice: ";
```

PROJECT

```
cin >> option;

cin.ignore();

switch (option) {

    case 1:

        addBook();

        break;

    case 2:

        displayBooks();

        break;

    case 3:

        searchBookByID();

        break;

    case 4:

        deleteBook();

        break;

    case 5:

        cout << "Exiting the system..." << endl;

        break;

    default:

        cout << "Invalid choice! Please try again." << endl;

}

} while (option != 5);

return 0;

}
```

PROJECT
